

Parallel Random Number Generation

Ashok Srinivasan
Florida State University
asriniva@cs.fsu.edu

- If random numbers were really random, then parallelization would not make any difference
 - ... and this talk would be unnecessary
 - But we use *pseudo*-random numbers, which only pretend to be random, and this causes problems
 - These problems can usually be solved if you use SPRNG!

Outline

- Introduction
- Random Numbers in Parallel Monte Carlo
- Parallel Random Number Generation
- SPRNG Libraries
- Conclusions

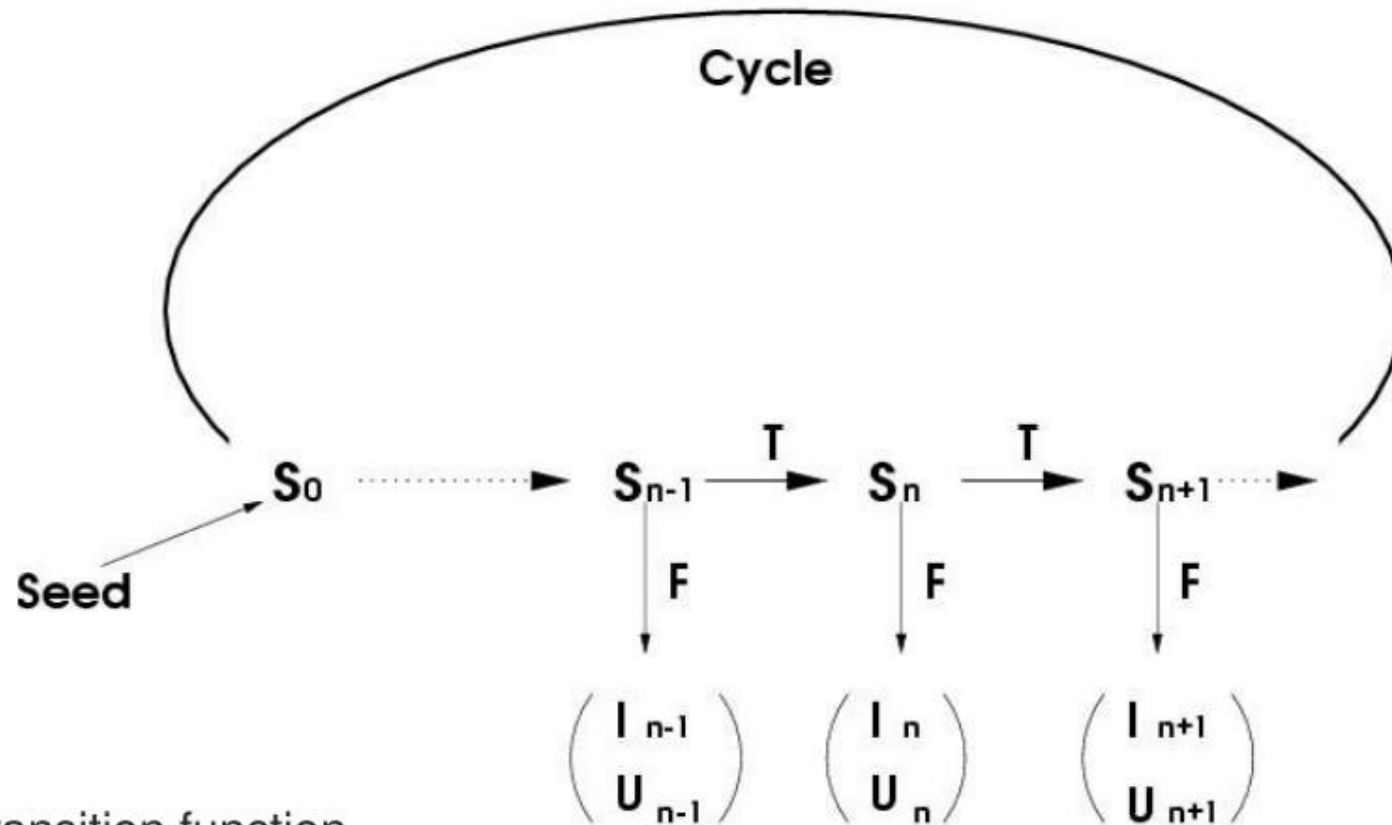
Introduction

- Applications of Random Numbers
- Terminology
- Desired Features
- Common Generators
- Errors Due to Correlations

Applications of Random Numbers

- Multi-dimensional integration using Monte Carlo
 - An important focus of this talk
 - Based on relating the expected value to an integral
- Modeling random processes
- Cryptography
 - Not addressed in this talk
- Games

Terminology



T: Transition function

Period: Length of the cycle

Desired Features

■ Sequential Pseudo-Random Number Generators

- Randomness
 - Uniform distribution in high dimensions
- Reproducibility
 - Helps in debugging
- Speed
- Large period
- Portability



Uniformity in 2-D

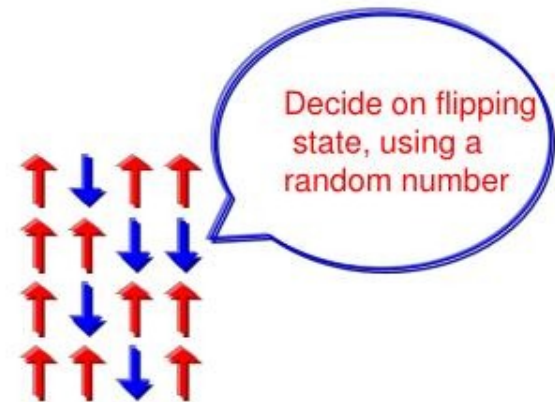
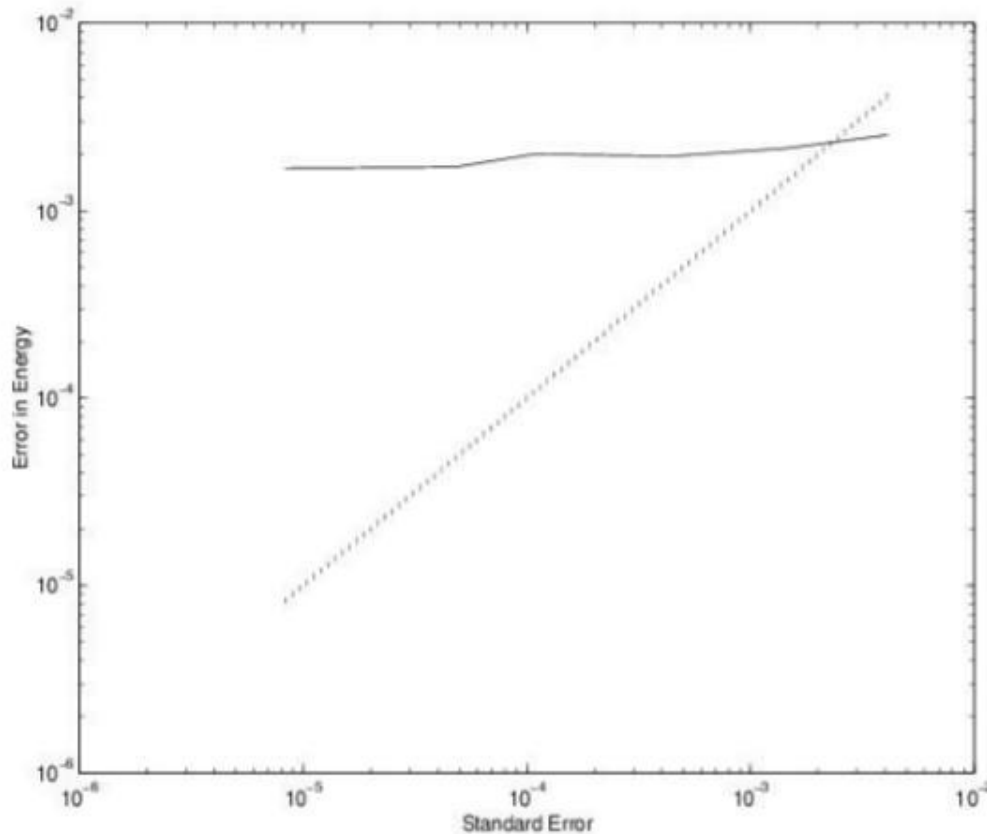
■ Parallel Pseudo-Random Number Generators

- Sequences on different processors should be uncorrelated
- Dynamic creation of new random number streams
- Absence of inter-processor communication

Common Generators

- Linear Congruential Generator (LCG)
 - $x_n = a x_{n-1} + p \pmod{m}$
- Additive Lagged Fibonacci Generator (LFG)
 - $x_n = x_{n-r} + x_{n-s} \pmod{m}$
- Multiple Recursive Generator (MRG)
 - Example: $x_n = a x_{n-1} + b x_{n-5} \pmod{m}$
 - Combined Multiple Recursive Generators (CMRG) combine multiple such generators
- Multiplicative Lagged Fibonacci Generator (MLFG)
 - $x_n = x_{n-r} \times x_{n-s} \pmod{m}$
- Mersenne Twister, etc

Error Due to Correlations

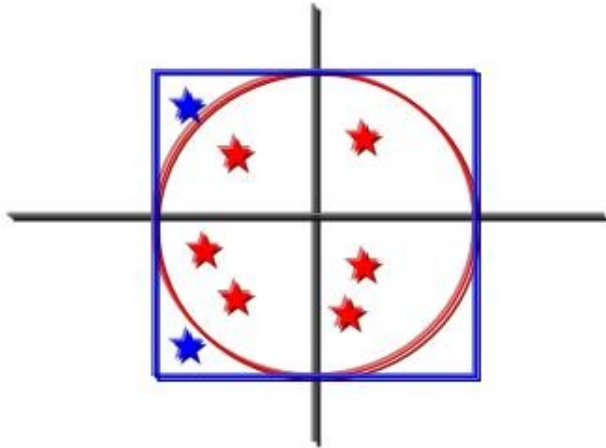


- Ising model results with Metropolis algorithm on a 16 x 16 lattice using the LFG *random*
 - The error is usually estimated from the standard deviation (x-axis), which should decrease as $(\text{sample size})^{-1/2}$

Random Numbers in Parallel Monte Carlo

- Monte Carlo Example: Estimating π
- Monte Carlo Parallelization
- Low Discrepancy Sequences

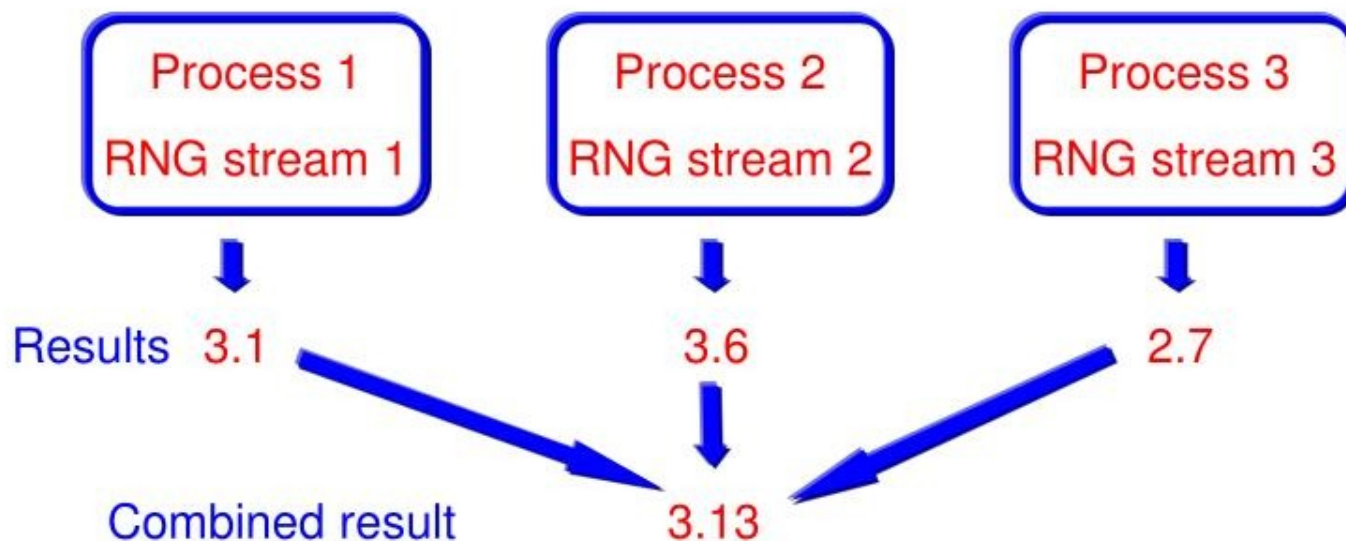
Monte Carlo Example: Estimating π



Uniform in 1-D but not in 2-D

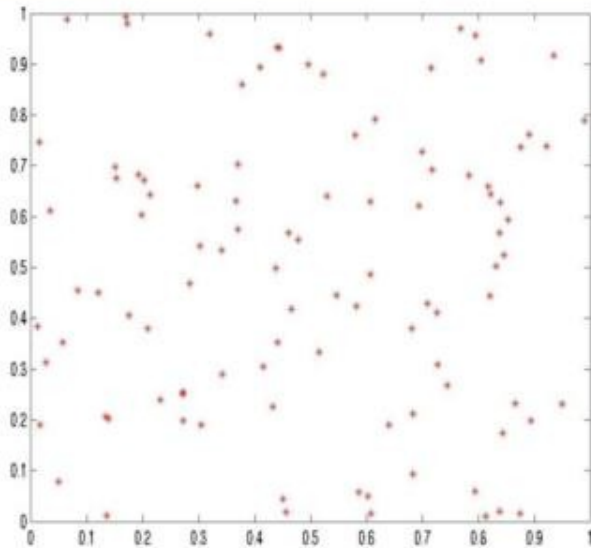
- Generate pairs of random numbers (x, y) in the square
 - Estimate π as: $4 \text{ (Number in circle) / (Total number of pairs)}$
 - This is a simple example of Monte Carlo integration
- Monte Carlo integration can be performed based on the observation that $E f(x) = \int f(y) \rho(y) dy$, where x is sampled from the distribution ρ
 - With N samples, $\text{error} \propto N^{-0.5}$
 - Example: $\rho = 1/4$, $f(x) = 1$ in the circle, and 0 outside, to estimate $\pi/4$

Monte Carlo Parallelization

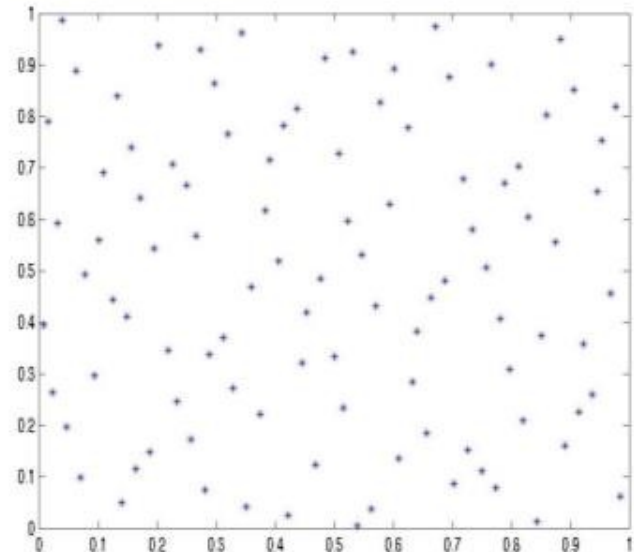


- Conventionally, Monte Carlo is “embarrassingly parallel”
 - Same algorithm is run on each processor, but with different random number sequences
 - For example, run the same algorithm for computing π
 - Results on the different processors can be combined together

Low Discrepancy Sequences



Random



Low Discrepancy Sequence

- Uniformity is often more important than randomness
 - Low discrepancy sequences attempt to fill a space uniformly
 - Integration error can be bound: $\propto \log^d N / N$, with N samples in d dimensions
 - Low discrepancy point sets can be used when the number of samples is known

Parallel Random Number Generation

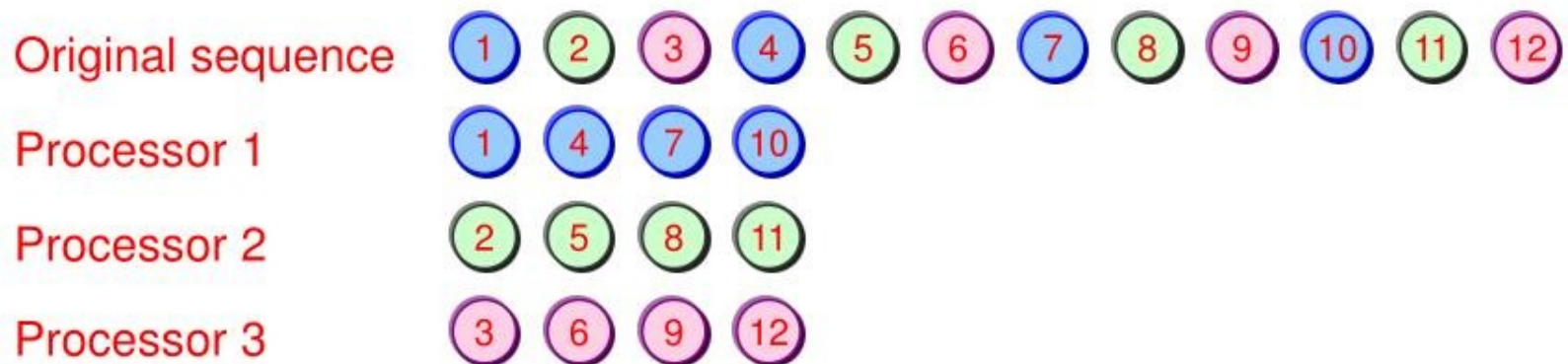
- Parallelization through Random Seeds
- Leap-Frog Parallelization
- Parallelization through Blocking
- Parameterization
- Test Results

Parallelization through Random Seeds

- Consider a single random number stream
- Each processor chooses a start state randomly
 - Hope that each start state is sufficiently far apart in the original stream
- Overlap of sequences possible, if the start states are not sufficiently far apart
- Correlations between sequences possible, even if the start states are far apart

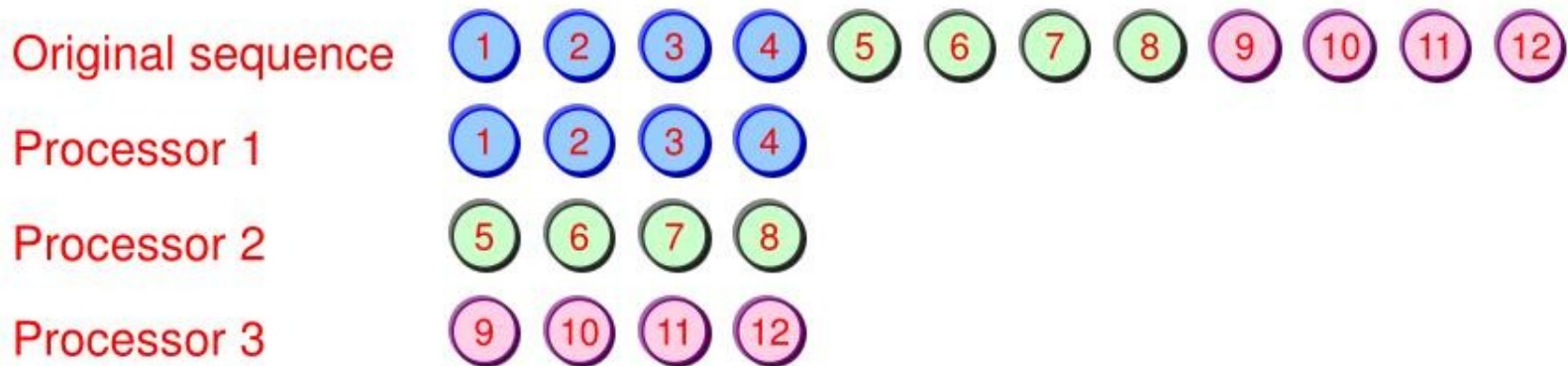
Leap-Frog Parallelization

- Consider a single random number stream
- On P processors, split the above stream by having each processor get every P^{th} number from the original stream
- Long-range correlations in the original sequence can become short-range intra-stream correlations, which are dangerous



Parallelization through Blocking

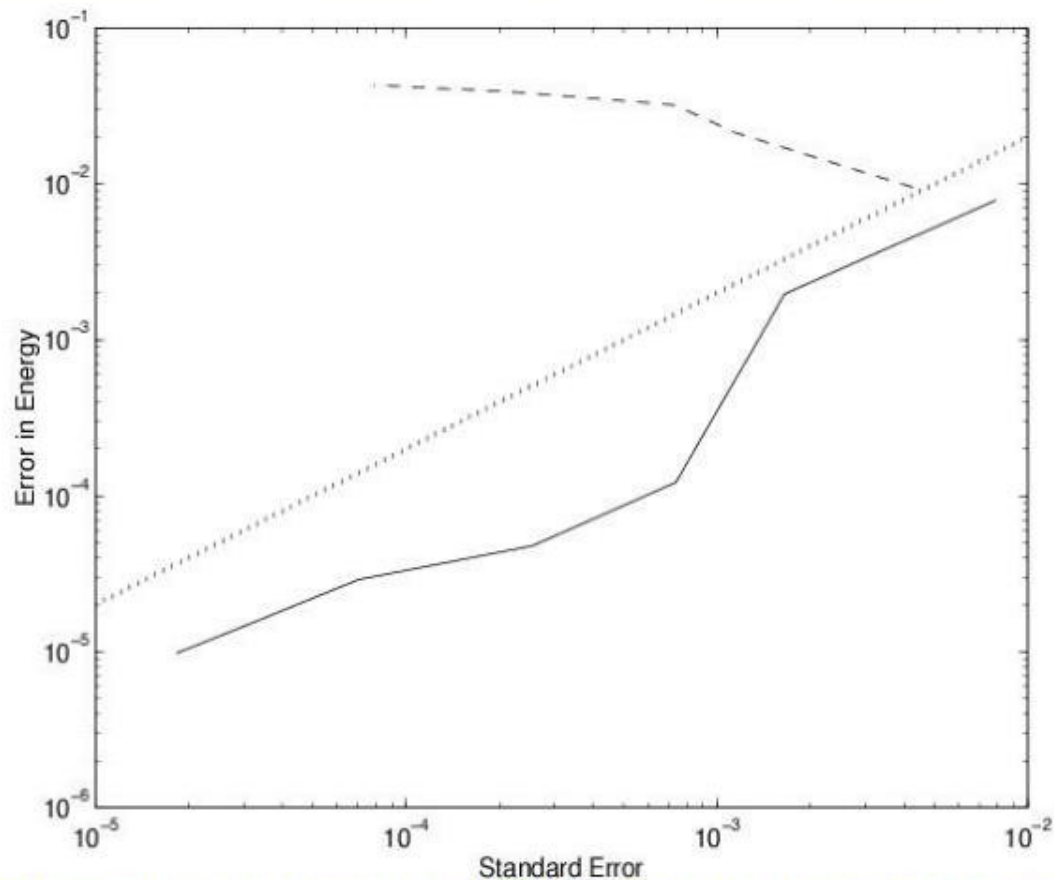
- Each processor gets a different block of numbers from an original random number stream
 - Long-range correlations in the original sequence can become short-range inter-stream correlations, which may be harmful
 - Example: The 48-bit LCG *ranf* fails the blocking test (add many numbers and see if the sum is normally distributed) with 10^{10} random numbers
 - Sequences on different processors may overlap



Parameterization

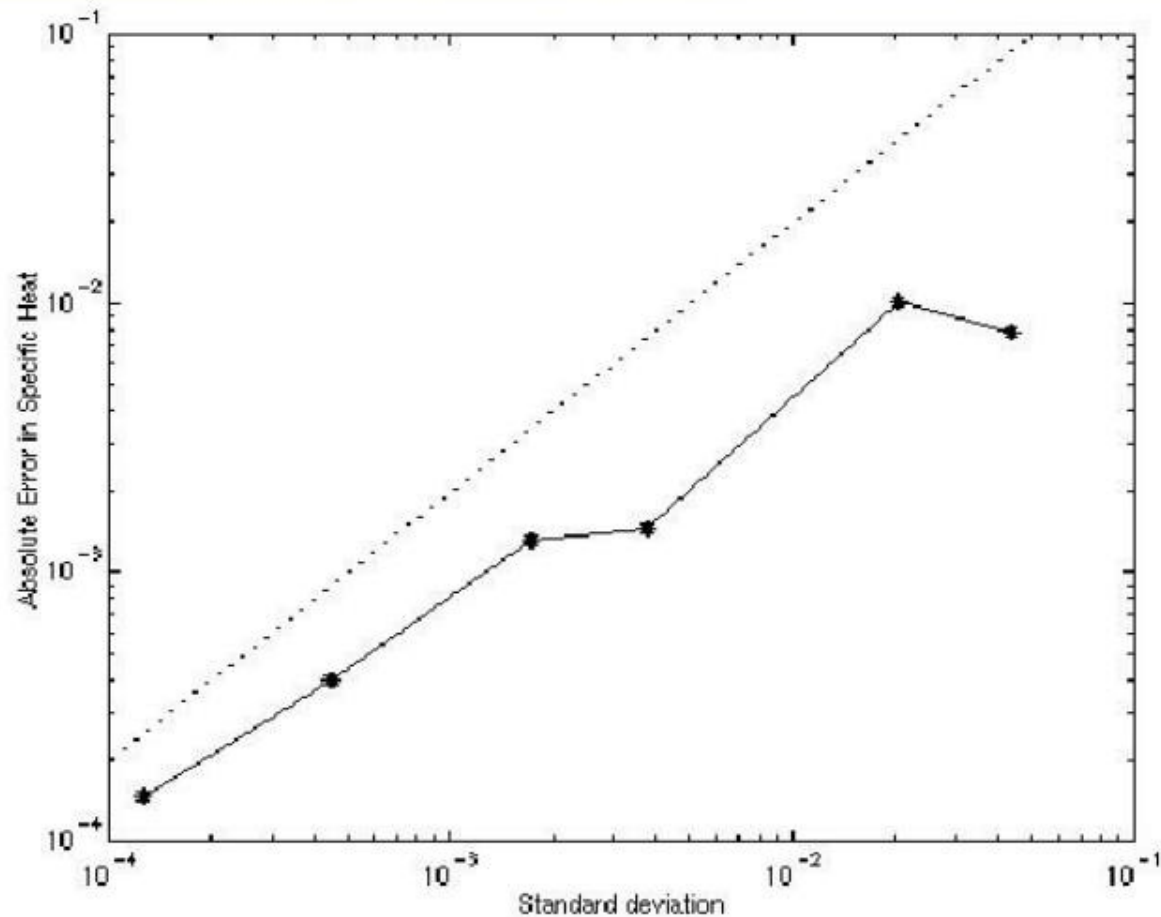
- Each processor gets an inherently different stream
- Parameterized iterations
 - Create a collection of iteration functions
 - Stream i is associated with iteration function i
 - LCG example: $x_n = a x_{n-1} + p_i \pmod{m}$ on processor i
 - p_i is the i^{th} prime
- Cycle parameterization
 - Some random number generators inherently have a large number of distinct cycles
 - Ensure that each processor gets a start state from a different cycle
 - Example: LFG
- The existence of inherently different streams does not imply that the streams are uncorrelated

Test Results 1



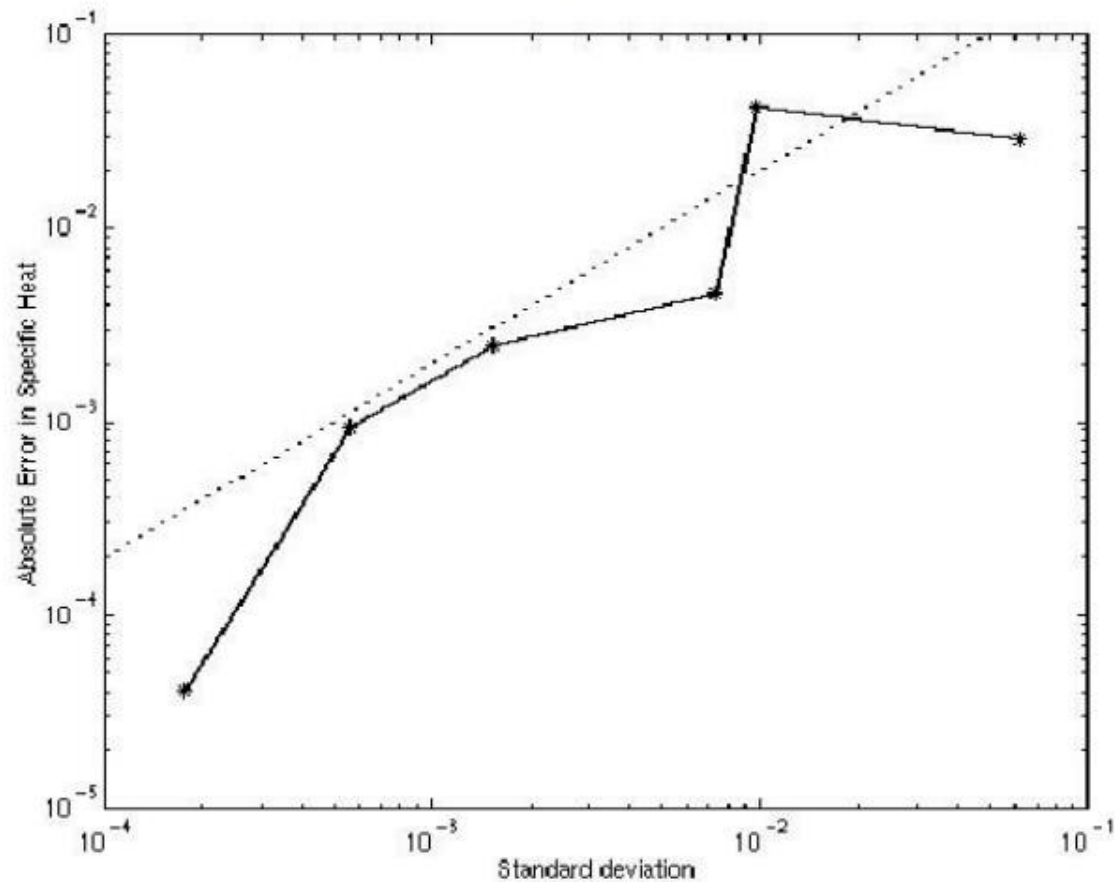
- Ising model results with Metropolis algorithm on a 16×16 lattice using a parallel LCG with (i) identical start states (dashed line) and (ii) different start states (solid line), at each site
 - Around 95% of the points should be below the dotted line

Test Results 2



- Ising model results with Metropolis algorithm on a 16×16 lattice using a sequential MLFG

Test Results 3



- Ising model results with Metropolis algorithm on a 16×16 lattice using a parallel MLFG

SPRNG Libraries

- SPRNG Features
- Simple Interface
- General Interface
- Spawning New Streams
- Test Suite
- Test Results Summary
- SPRNG Versions

SPRNG Features

- Libraries for parallel random number generation
 - Three LCGs, a modified LFG, MLFG, and CMRG
 - Parallelization is based on parameterization
 - Periods up to 2^{1310} , and up to 2^{39618} distinct streams
- Applications can dynamically spawn new random number streams
 - No communication is required
- PRNG state can be checkpointed and restarted in a machine independent manner
- A test suite is included, to enable testing the quality of parallel random number generators
- An extensibility template enables porting new generators into SPRNG format
- Usable in C/C++ and Fortran programs

Simple Interface

```
#include <stdio.h>

#define SIMPLE_SPRNG
#include "sprng.h"

main()
{
    double rn;  int i;

    printf(" Printing 3 random numbers in
           [0,1):\n");
    for (i=0;i<3;i++)
    {
        rn = sprng(); /* double precision */
        printf("%f\n",rn);
    }
}
```

```
#include <stdio.h>
#include <mpi.h>
#define SIMPLE_SPRNG
#define USE_MPI
#include "sprng.h"

main(int argc, char *argv[])
{
    double rn; int i, myid;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &myid);

    for (i=0;i<3;i++) {
        rn = sprng();
        printf("Process %d, random
               number %d: %.14f\n", myid, i+1,
               rn);
    }
    MPI_Finalize();
}
```

General Interface

```
#include <stdio.h>
#include <mpi.h>
#define USE_MPI
#include "sprng.h"

main(int argc, char *argv[])
{
    int streamnum, nstreams, seed,
        *stream, i, myid, nprocs;
    double rn;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &myid);
    MPI_Comm_size(MPI_COMM_WORLD,
        &nprocs);
    streamnum = myid;
    nstreams = nprocs;
    seed = make_sprng_seed();
```

```
    stream = init_sprng(streamnum,
        nstreams, seed, SPRNG_DEFAULT);
    for (i=0;i<3;i++)
    {
        rn = sprng(stream);
        printf("process %d, random number
            %d: %f\n", myid, i+1, rn);
    }

    free_sprng(stream);
    MPI_Finalize();
}
```

Spawning New Streams

- Can be useful in ensuring reproducibility
 - Each new entity is given a new random number stream

```
#include <stdio.h>
#include "sprng.h"
#define SEED 985456376
main()
{
    int streamnum, nstreams, *stream,
        **new;
    double rn;  int i, nspawned;

    streamnum = 0;
    nstreams = 1;
    stream = init_sprng(streamnum,
        nstreams, SEED, SPRNG_DEFAULT);
    for (i=0;i<20;i++)
        rn = sprng(stream);

    nspawned = spawn_sprng(stream, 2,
        &new);
```

```
    printf(" Printing 2 random numbers
        from second spawned stream:\n");
    for (i=0;i<2;i++)
    {
        rn = sprng(new[1]);
        printf("%f\n", rn);
    }

    free_sprng(stream);
    free_sprng(new[0]);
    free_sprng(new[1]);
    free(new);
}
```

Converting Code to Use SPRNG

```
#include <stdio.h>
#include <mpi.h>
#define SIMPLE_SPRNG
#define USE_MPI
#include "sprng.h"
#define myrandom sprng
double myrandom(); /* Old PRNG */

main(int argc, char *argv[])
{
    int seed, i, myid;  double rn;
    MPI_Init(&argc, &argv);
    for (i=0;i<3;i++)
    {
        rn = myrandom();
        printf("Process %d, random number %d: %.14f\n", myid, i+1, rn);
    }
    MPI_Finalize();
}
```

Test Suite

- Sequential and parallel tests to check for absence of correlations
 - Tests run on sequential or parallel machines
- Parallel tests interleave different streams to create a new stream
 - The new streams are tested with sequential tests

Original streams

Stream 1    ...

Stream 2    ...

Stream 3    ...

New stream          ...

Test Results Summary

- Sequential and parallel versions of DIEHARD and Knuth's tests
- Application-based tests
 - Ising model using Wolff and Metropolis algorithms, random walk test
- Sequential tests
 - 1024 streams typically tested for each PRNG variant, with a total of around $10^{11} - 10^{12}$ random numbers used per test per PRNG variant
- Parallel tests
 - A typical test creates four new streams by combining 256 streams for each new stream
 - A total of around $10^{11} - 10^{12}$ random numbers were used for each test for each PRNG variant
- All SPRNG generators pass all the tests
 - Some of the largest PRNG tests conducted

SPRNG Versions

- All the SPRNG versions use the same generators, with the same code used in SPRNG 1.0
 - The interfaces alone differ
- SPRNG 1.0: An application can use only one type of generator
 - Multiple streams can be used, of course
 - Ideal for the typical C/Fortran application developer, usable from C++ too
- SPRNG 2.0: An application can use multiple types of generators
 - There is some loss in speed
 - Useful for those developing new generators by combining existing ones
- SPRNG 4.0: C++ wrappers for SPRNG 2.0
- **SPRNG Cell**: SPRNG for the SPU's of the Cell processor
 - Available from Sri Sathya Sai University, India

Conclusions

- Quality of sequential and parallel random number generators is important in applications that use a large number of random numbers, or those that use several processors
 - Speed is probably less important, to a certain extent
- It is difficult to prove the quality, theoretically or empirically
 - Use different types of generators, verify if their results are similar using the individual solutions and the estimated standard deviation, and then combine the results if they are similar
- It is important to ensure reproducibility, to ease debugging
- Use SPRNG!
 - sprng.scs.fsu.edu